

Lab 2 — Network Setup

Understanding and Configuring Linux Networking

Table of contents

1	Introduction	1
2	Background: How IP Addressing Works	2
3	Using the Terminal	4
3.1	Checking Your Network Settings	4
3.1.1	Reading the Output	6
3.2	Manually Configuring Your Network Settings	8
3.3	Switching Back to Louisiana Tech's Network	10
4	The Point-and-Click Way (Linux Mint)	11
5	Summary	13

1 Introduction

Before you can attack or defend a networked system, you need to understand how networks are configured at a low level. In this lab you will learn to read your current network settings, manually assign an IP address from the terminal, and restore your original connection — all skills that are essential for the Cyber Storm competition.

Key Concepts at a Glance

Term	What It Means
IP Address	A unique numerical label identifying a device on a network
Netmask / Subnet Mask	Defines which portion of an IP address is the network vs. the host
CIDR Notation	A compact way to write a network address and its mask (e.g. 10.0.0.0/8)
DHCP	A protocol that automatically assigns IP addresses to devices
NAT	Network Address Translation — a technique that lets many devices share one public IP
Loopback	A virtual interface (10) that routes packets back to the same machine
Interface	A network adapter — physical (Ethernet, Wi-Fi) or virtual

2 Background: How IP Addressing Works

IP Addresses and Subnets

Every device on an IP network needs three pieces of information to communicate:

1. IP Address A 32-bit number (for IPv4) written as four decimal octets separated by dots, e.g. 138.47.166.224. This uniquely identifies your machine on the network.

2. Subnet Mask (Netmask) Also a 32-bit number, written the same way, e.g. 255.255.248.0. The mask divides the IP address into two parts:

- The **network portion** — bits covered by the 255 octets — identifies the network your machine belongs to. All devices on the same network share this prefix.
- The **host portion** — bits covered by the 0 octets — identifies your specific machine within that network.

For example, with the address 138.47.166.224 and mask 255.255.248.0:

- Network: 138.47.160.0 (the first 21 bits are fixed)

- Host: 0.0.6.224 (the last 11 bits are yours)
- Valid host range on this subnet: 138.47.160.1 – 138.47.167.254

3. Broadcast Address The last address in a subnet (138.47.167.255 above). A packet sent to the broadcast address is delivered to every device on the subnet.

i CIDR Notation

Typing 255.255.248.0 is cumbersome. **CIDR (Classless Inter-Domain Routing)** notation expresses the same information more concisely by appending a / and the number of network bits to the IP address.

255.255.248.0 has 21 consecutive 1 bits, so it is written as /21.

138.47.166.224/21	↔	IP: 138.47.166.224, Mask: 255.255.248.0
10.0.0.0/8	↔	IP: 10.0.0.0, Mask: 255.0.0.0
192.168.1.0/24	↔	IP: 192.168.1.0, Mask: 255.255.255.0

The /8, /24, and /21 tell you how many bits belong to the network — and therefore how large the subnet is (how many hosts it can hold).

i DHCP — Dynamic Host Configuration Protocol

When you connect a device to a network, it usually does not have an IP address yet. **DHCP** solves this automatically:

1. Your device broadcasts a *DHCP Discover* message: “Is there a DHCP server here?”
2. A DHCP server on the network replies with a *DHCP Offer*: “Here, use this IP address.”
3. Your device accepts the offer (*DHCP Request*) and the server confirms (*DHCP Ack*).

The server also provides the **subnet mask**, **default gateway**, and **DNS servers** as part of the same exchange. This entire handshake happens in milliseconds every time you connect to a network.

In this lab you will bypass DHCP and configure an address manually — this is called a **static** IP configuration.

i NAT vs. Bridged Adapter (VirtualBox)

When running a virtual machine (VM), the way VirtualBox connects it to the network matters enormously:

NAT (Network Address Translation)

VirtualBox acts as a mini-router between your VM and the outside world. Your VM gets a private address (usually in the 10.X.Y.Z range) that only VirtualBox can see. The host machine translates packets on the VM's behalf. The VM can reach the internet, but other machines on the real network *cannot reach the VM* by its IP — because that IP is not visible outside the host.

Bridged Adapter

VirtualBox bridges the VM's virtual network card directly onto your physical network. The VM gets a real IP address from your network's DHCP server (e.g. 138.47.X.Y on the Louisiana Tech network) and is visible to other machines just like a physical device. This is required for Cyber Storm, where your VM must be reachable by other competitors on the same subnet.

Mode	VM gets real network IP	Other machines can reach VM	Internet access
NAT	no (private 10.x)	no	yes
Bridged	yes	yes	yes

Note: this section and the rest of this lab assume you are using VirtualBox. If you are on Apple Silicon and using UTM instead (see the callout in Lab 1), the underlying concepts still apply, but the menu names and steps below will not match — see Lab 1 for a disclaimer on this.

3 Using the Terminal

3.1 Checking Your Network Settings

To inspect your current network configuration, run:

```
ip addr show
```

(`ip a` is a shorthand for the same command.)

i A Note on `ifconfig`

You will see `ifconfig` used in a lot of older tutorials, documentation, and some competition/CTF environments. It is part of the older `net-tools` package and is considered

legacy — the `ip` command (from `iproute2`) is its modern replacement and is installed by default on current Linux distributions, while `net-tools` increasingly is not. This lab uses `ip` throughout, but if you ever need `ifconfig` on an older or unfamiliar system, you can install it with:

```
sudo apt install net-tools
```

We'll also note the equivalent `ifconfig` command alongside each `ip` command below, so you can recognize it if you run into it elsewhere.

The output will look something like this:

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
↪  default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
↪  group default qlen 1000
   link/ether 08:00:27:75:a7:90 brd ff:ff:ff:ff:ff:ff
   inet 138.47.166.224/21 brd 138.47.167.255 scope global enp0s8
       valid_lft forever preferred_lft forever
   inet6 fe80::a00:27ff:fe75:a790/64 scope link
       valid_lft forever preferred_lft forever
```

💡 The `ifconfig` Equivalent

If you were using `ifconfig` instead, the same interface would look like this:

```
enp0s8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 138.47.166.224  netmask 255.255.248.0  broadcast
        ↪  138.47.167.255
        inet6 fe80::a00:27ff:fe75:a790  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:75:a7:90  txqueuelen 1000  (Ethernet)
```

The main difference: `ifconfig` shows the netmask separately (255.255.248.0) instead of combined into CIDR notation (/21), and labels the MAC address `ether` instead of `link/ether`.

3.1.1 Reading the Output

There are two entries listed: 2: `enp0s8` and 1: `lo`.

lo — the Loopback Interface

The loopback interface is a virtual interface that routes packets straight back to your own machine without sending anything onto the physical network. It always has the address `127.0.0.1` (or `::1` for IPv6). You will see this used as `localhost` in many contexts — for example, when you ran `ftp localhost` or `ssh localhost` in Lab 3. It is mainly used for local testing and inter-process communication.

enp0s8 — the Ethernet Interface

This is the interface connected to the real network. **Your interface label may differ** — common alternatives include `eth0`, `enp0s3`, or `ens33`. The naming convention for modern Linux interface names like `enp0s8` has three parts:

Part	Meaning
en	Ethernet
p0	PCI bus 0
s8	Slot 8 on that bus

The key fields for this interface are:

```
2: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
    ↪ group default qlen 1000
    link/ether 08:00:27:75:a7:90 brd ff:ff:ff:ff:ff:ff
    inet 138.47.166.224/21 brd 138.47.167.255 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe75:a790/64 scope link
```

Field	Value (example)	What It Means
state	UP	Whether the interface is administratively up and has a carrier signal
link/ether	08:00:27:75:a7:90	MAC address — a hardware identifier burned into the NIC
inet	138.47.166.224/21	Your IPv4 address, already combined with its subnet mask in CIDR notation

Field	Value (example)	What It Means
<code>brd</code>	<code>138.47.167.255</code>	Broadcast address used to reach all hosts on your subnet
<code>scope global</code>	—	This address is routable beyond the local machine (vs. <code>scope link</code> for link-local addresses)
<code>inet6</code>	<code>fe80::a00:...</code>	Your IPv6 link-local address (auto-generated)

Notice that `ip addr show` gives you the netmask for free as part of the `inet` line (`/21`), instead of spelling it out separately the way `ifconfig` does — another place where the CIDR notation from earlier pays off. If you want packet/byte counters like `ifconfig` shows, use `ip -s link show enp0s8` instead.

The address `138.47.166.224/21` was assigned automatically by a **DHCP** server on Louisiana Tech's network.

Troubleshooting: IP Starts with 10.X.Y.Z?

If your IP address begins with 10, your VM is in **NAT** mode. Other machines on the network cannot reach you, which will prevent you from participating in Cyber Storm.

Fix (VirtualBox):

If you are using UTM instead of VirtualBox (see the Apple Silicon callout in Lab 1), the steps below won't match — look for the equivalent network mode setting in UTM's VM configuration.

1. Power off the VM (do **not** save the machine state).
2. In the VirtualBox main window, select the VM and click **Settings**.
3. Go to **Network** → **Adapter 1**.
4. Change *Attached to* from **NAT** to **Bridged Adapter**.
5. Under *Name*, select your **Wi-Fi** adapter (often labeled with 802.11 or WiFi).
6. Boot the VM back up and run `ip addr show` again.

On Louisiana Tech's network the address should now begin with `138.47`. At home it will typically be `192.168.1.X`.

3.2 Manually Configuring Your Network Settings

For Cyber Storm, competitors are placed on the private 10.0.0.0/8 network and must manually assign their own static IP address. DHCP is not available in the competition environment.

Why Manual Configuration?

In competition and penetration testing scenarios, networks are often isolated from the internet and have no DHCP server. Being able to assign a static IP from memory — without a GUI — is a fundamental sysadmin and security skill. It also gives you precise control: you can place yourself on any subnet you choose, which matters when you need to communicate with specific target machines.

Suppose you want to assign your machine the IP address 10.4.5.6 with a /8 netmask. Run the following command, replacing `enp0s8` with your actual interface name:

```
sudo ip addr add 10.4.5.6/8 dev enp0s8
sudo ip link set enp0s8 up
```

Breaking this down:

Part	Meaning
<code>sudo</code>	Run with administrator privileges — required to change network settings
<code>ip addr add</code> <code>10.4.5.6/8</code>	Add a new address to an interface The IP address and CIDR mask to assign (/8 = 255.0.0.0 — all 10.X.Y.Z addresses are on the same network)
<code>dev enp0s8</code> <code>ip link set enp0s8 up</code>	Target this specific network interface Ensure the interface is administratively up (usually already the case, but doesn't hurt to confirm)

The `ifconfig` Equivalent

```
sudo ifconfig enp0s8 10.4.5.6 netmask 255.0.0.0 up
```

Verify the change by running `ip addr show` again and confirming the new address appears for your interface.

💡 Testing That It Worked

If you are doing this during the network setup lab, try visiting a website like [Google](#) in a browser. If the page **does not load**, that is expected and correct — you have moved your machine off Louisiana Tech’s network and onto the isolated 10.0.0.0/8 network, which has no internet route.

If the page **does** load, double-check that the IP address changed as expected with `ip addr show`.

⚠️ Don’t Forget Your Interface Name

Every command in this section that includes `enp0s8` must use **your actual interface name**. Run `ip addr show` first and note the label of your Ethernet interface before proceeding.

🔥 Troubleshooting: Old Address Won’t Go Away / Internet Still Works

If you run the commands above and `ip addr show` still shows your original 138.47.X.Y address sitting alongside the new 10.4.5.6, or the internet still loads when it shouldn’t, this almost always means **NetworkManager is still managing the interface**. `ip addr add` only *adds* an address — it does not remove the DHCP one — and NetworkManager will keep renewing/reasserting the original address in the background, so the two commands from earlier aren’t enough on their own.

Check whether NetworkManager owns the interface:

```
nmcli device status
```

If your interface shows as **connected** (not **unmanaged**), tell NetworkManager to let go of it, then flush and reassign:

```
sudo nmcli device set enp0s8 managed no
sudo ip addr flush dev enp0s8
sudo ip addr add 10.4.5.6/8 dev enp0s8
sudo ip link set enp0s8 up
```

Run `ip addr show` again — you should now see **only** 10.4.5.6/8 on the interface.

3.3 Switching Back to Louisiana Tech's Network

When you are done with the competition or lab exercise, restore your connection to Tech's network by resetting the interface and requesting a new address from DHCP. Run the following three commands in order:

Step 1 — Remove the current IP address and bring the interface down:

```
sudo ip addr flush dev enp0s8
sudo ip link set enp0s8 down
```

💡 The `ifconfig` Equivalent

```
sudo ifconfig enp0s8 0.0.0.0 down
```

Step 2 — Bring the interface back up (with no address yet):

```
sudo ip link set enp0s8 up
```

💡 The `ifconfig` Equivalent

```
sudo ifconfig enp0s8 up
```

Step 3 — Request a new IP address from the DHCP server:

```
# On Linux Mint (managed by NetworkManager):
sudo nmcli device connect enp0s8

# On Ubuntu Server (managed by systemd-networkd):
sudo systemctl restart systemd-networkd
```

💡 Which command do I use in Step 3?

- If you are running **Linux Mint** (or any Ubuntu desktop variant, which normally uses NetworkManager), use `nmcli`.
- If you are running **Ubuntu Server** (`systemd-networkd`, no NetworkManager), use the `systemctl` command.

Not sure which one manages your network? Run `nmcli device status`. If it lists your interface, use the `nmcli` command above. If `nmcli` is not installed, you're most likely on

a systemd-networkd-only system — use the `systemctl` command instead.

🔥 Troubleshooting: `nmcli device connect` Doesn't Bring Tech's Network Back

If you followed the “Troubleshooting” callout earlier in this lab and ran `sudo nmcli device set enp0s8 managed no`, you must hand the interface back to NetworkManager **before** Step 3 will do anything:

```
sudo nmcli device set enp0s8 managed yes
```

Even after this, `nmcli device connect` sometimes will not force a fresh DHCP request — NetworkManager may still be holding onto stale connection state. If `ip addr show` still doesn't show a `138.47.X.Y` address after Step 3, force a clean disconnect/reconnect cycle instead:

```
sudo nmcli device disconnect enp0s8
sudo nmcli device connect enp0s8
```

This is the CLI equivalent of clicking the wired connection icon in the system tray to disconnect and reconnect, which also works if you'd rather use the GUI.

ℹ The `dhclient` Equivalent

Older systems (or ones without NetworkManager) may instead rely on the ISC DHCP client, `dhclient`. It's often not installed by default anymore, so install it first if needed:

```
sudo apt install isc-dhcp-client # if dhclient is not already available
sudo dhclient enp0s8
```

After running these commands, verify your address has returned to the `138.47.X.Y` range (on campus) or `192.168.X.X` (at home) by running `ip addr show`, then confirm internet access by visiting a website.

4 The Point-and-Click Way (Linux Mint)

Everything above used the terminal. Linux Mint's Cinnamon desktop also ships a graphical tool for the same job — useful if you'd rather point-and-click, or if you're troubleshooting on

a machine where the terminal approach isn't cooperating.

i When to Use This

The GUI tool and the `ip/nmcli` commands used earlier in this lab all talk to the same NetworkManager service under the hood, so the same caveats apply. In particular, if you previously ran `sudo nmcli device set enp0s8 managed no` while troubleshooting, set it back to `managed yes` first — NetworkManager (and this GUI tool) can't control an unmanaged interface.

Step 1: Open Edit Connections

Right-click the network icon in the system tray (bottom-right of the panel) and select **Edit Connections...**. If that option isn't available, open the **Menu** and search for **Network Connections**, or launch the tool directly:

```
nm-connection-editor
```

Step 2: Edit your wired connection

Select the connection for your interface (usually listed as **Wired connection 1**, or named after `enp0s8/eth0/etc.`) and click the gear icon to edit it.

Step 3: Switch to Manual IPv4 configuration

Open the **IPv4 Settings** tab and change the **Method** dropdown from **Automatic (DHCP)** to **Manual**.

Step 4: Add the address

Click **Add** and fill in the fields, using the same values from the command-line example earlier in this lab:

Field	Value
Address	10.4.5.6
Netmask	255.0.0.0 (some versions of the editor instead want the CIDR prefix, 8)
Gateway	leave blank unless your subnet requires one

Click **Save**.

💡 Netmask vs. Prefix Length

If the Netmask field rejects 255.0.0.0, try 8 instead (or vice versa) — this varies by Mint/Cinnamon version.

Step 5: Reconnect the connection

Saving the profile doesn't apply it to an already-active connection — you have to reconnect. Click the network icon in the system tray, turn the wired connection off, then back on. This is the GUI equivalent of the `nmcli device disconnect / nmcli device connect` cycle used earlier in this lab.

Step 6: Verify

Confirm the change from the terminal, the same way you did for the command-line method:

```
ip addr show enp0s8
```

You should see 10.4.5.6/8 listed for your interface, in place of the original 138.47.X.Y address.

🔥 Switching Back

To return to DHCP later, reopen **Edit Connections**, go back to **IPv4 Settings**, and change **Method** back to **Automatic (DHCP)**. Save, then disconnect/reconnect the connection as in Step 5 to request a fresh 138.47.X.Y address.

5 Summary

Here is the full sequence of what you can do in this lab, end to end:

Task	Command	Legacy (ifconfig/dhclient) equivalent
View network interfaces	<code>ip addr show</code>	<code>ifconfig</code>
Assign a static IP	<code>sudo ip addr add <ip>/<cidr> dev <iface></code>	<code>sudo ifconfig <iface> <ip> netmask <mask> up</code>

Task	Command	Legacy (<code>ifconfig</code> / <code>dhclient</code>) equivalent
Take an interface down	<code>sudo ip addr flush dev <iface></code> then <code>sudo ip link set <iface> down</code>	<code>sudo ifconfig <iface> 0.0.0.0 down</code>
Bring an interface up	<code>sudo ip link set <iface> up</code>	<code>sudo ifconfig <iface> up</code>
Request a DHCP address (Mint)	<code>sudo nmcli device connect <iface></code>	<code>sudo dhclient <iface></code>
Request a DHCP address (Ubuntu Server)	<code>sudo systemctl restart systemd-networkd</code>	—

The legacy tools aren't installed by default on current systems — see the “A Note on `ifconfig`” and “The `dhclient` Equivalent” callouts above for the install commands if you need them on an older machine.

💡 Going Further

- **Subnet math:** Practice converting between netmasks and CIDR notation. Given `192.168.10.50/26`, can you work out the network address, broadcast address, and valid host range?
- **`ifconfig` command:** Now that you've used `ip`, install `net-tools` and try the `ifconfig` equivalents from the callouts above. You'll likely still run into `ifconfig` on older systems, in older tutorials, and in some CTF/competition environments, so it's worth being able to read.
- **Wireshark:** Capture a DHCP handshake by triggering a new DHCP request (e.g. `sudo nmcli device connect enp0s8` or `sudo dhclient enp0s8`) while Wireshark is recording. You will see the four-step Discover → Offer → Request → Ack exchange in real time.
- **Static persistence:** Changes made with `ip` are temporary and lost on reboot. Research how to make a static IP address permanent via Netplan (`/etc/netplan/*.yaml`) on Ubuntu or the Network Manager on Mint.